

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Big Data Analytics and Business Intelligence

Group 5 Project Report

Big data in Manufacturing: Predictive Maintenance for Bearing

Advisor(s): Assoc. Prof. Vo Thi Ngoc Chau

Student(s): Nguyen Trinh Chau 2252091

Trinh Duc Tam 2252734

Khuu Vinh Kien 2211720

HO CHI MINH CITY, JANUARY 2026



Contents

1	Introduction	6
1.1	Motivation	6
1.2	Goal	6
1.3	Scope	7
2	Feature Engineering and Predictive Model	8
2.1	Dataset overview	8
2.2	Feature Engineering	10
2.2.1	Candidate Feature Extraction	10
2.2.2	Correlation Analysis	11
2.3	Predictive Model	12
2.3.1	Candidate Models	12
2.3.2	Model Selection	13
2.3.3	Gradient Boosting Theory	14
3	System architecture	15
3.1	Architecture Overview	15
3.2	Detailed Module Design	16
3.2.1	Data Ingestion	16
3.2.2	Processing Engine (The Core)	16
3.2.3	Storage Layer	16
3.2.4	Visualization & Monitoring Layer	16
4	Technology Stack	17
4.1	Data Ingestion	17
4.2	Processing	17
4.3	Storage	18
4.4	Dashboard	18
4.5	Infrastructure and Deployment	19
5	Implementation	20
5.1	Data Ingestion	20
5.2	Processing and Storage	22
5.2.1	Batch Processing (Training Model)	22
5.2.2	Stream Processing And Storage	23



- 5.3 Dashboard Integration 27
- 6 System Demonstration & Results 28**
 - 6.1 Overview Page 28
 - 6.1.1 Feature 2: Executive Summary 28
 - 6.1.2 Feature 2: Fleet Health Matrix (Diagnostic Scatter Plot) 29
 - 6.2 Vibration History and RUL Prediction 30
 - 6.2.1 Feature 3: Vibration History and Trend 30
 - 6.2.2 Alert History Log 31
 - 6.2.3 Feature 5: Remaining Useful Life (RUL) Gauge 31
 - 6.3 Prescriptive Analytics: Risk-Reward Optimization 31
 - 6.3.1 Feature 6: Effective Plan 31
- 7 Conclusion & Future Work 34**
 - 7.1 Conclusion 34
 - 7.2 Limitations 34
 - 7.3 Future Work 34
- 8 Video and Source Code 36**



List of Figures

2.1	Dataset Experiment Setup	8
2.2	Dataset Overview	9
2.3	Dataset Overview	12
3.1	System Architecture	15
6.1	Executive summary page	28
6.2	Vibration History and RUL Page	30
6.3	Maintenance Risk-Reward Matrix for Economic Optimization	32

1 Introduction

1.1 Motivation

In the modern industrial landscape, mechanical systems are the backbone of production lines, and among their various components, bearings are considered the "heart" of rotating machinery. These precision-engineered components are designed to support rotating shafts, reduce friction between moving parts, and withstand significant radial and axial loads. From small electric motors to massive industrial turbines, bearings are ubiquitous, ensuring that energy is transmitted efficiently with minimal wear.

Bearings operate under constant stress and high-speed conditions, their physical integrity directly dictates the reliability of the entire manufacturing process. In a globalized economy where "Just-In-Time" production is the standard, even a minor bearing defect can lead to increased vibration, reduced product quality, and eventually, catastrophic equipment failure. Consequently, monitoring the health of bearings is not merely a maintenance task but a critical strategic requirement to ensure industrial productivity and operational safety.

Predictive Maintenance for bearings emerges as the optimal solution by utilizing real-time sensor data to intervene only when necessary. By detecting early signs of degradation—such as anomalies in vibration patterns—factories can transition from a "fail-and-fix" mindset to a "predict-and-prevent" strategy. This transition is crucial for minimizing downtime, maximizing the lifecycle of mechanical assets, and significantly reducing maintenance overhead in high-output manufacturing environments.

1.2 Goal

The primary objective of this project is to develop a comprehensive system that bridges the gap between raw sensor data and actionable business intelligence. Our goals are categorized into three core pillars:

- **Build an end-to-end real-time monitoring pipeline:** Establish a robust data flow using Apache Kafka and Spark to ingest and process high-frequency vibration data as it is generated.
- **Build an AI model to predict Remaining Useful Life (RUL):** Implement a Machine Learning regressor capable of analyzing degradation trends and estimating the number of cycles or hours remaining before a bearing reaches its failure threshold.



- **Create an interactive dashboard for decision support:** Design a visual interface that provides real-time health scores and RUL predictions, enabling maintenance teams to make data-driven decisions regarding equipment intervention.

1.3 Scope

To ensure a high-quality implementation, the scope of this project is confined to the following operational boundaries:

- **Deployment Environment:** The system is designed and executed on a single local workstation environment for demonstration purposes, rather than being deployed across a multi-node distributed hardware cluster.
- **System Architecture:** The implementation focuses on a centralized setup that demonstrates the full integration of data ingestion, stream processing, and persistent storage.
- **Operational Scale:** The project simulates real-time data flow using historical records to validate the pipeline's logic and the predictive model's accuracy.
- **Final Output:** The project concludes with a functional interactive dashboard designed for data visualization and executive decision support. The development of an external cloud-based web application or public-facing website is beyond the current scope of this work.

2 Feature Engineering and Predictive Model

In this section we will introduce our proposed solution for bearing maintainance. We will discuss about our system architecture predictive model and feature engineering.

2.1 Dataset overview

First we would like to introduce our dataset. For our project, we decided to choose NASA Bearing Dataset. For the dataset, the data was generated by the NSF I/UCR Center for Intelligent Maintenance Systems with support from Rexnord Corp. in Milwaukee, WI.

Four bearings were installed on a shaft. The rotation speed was kept constant at 2000 RPM by an AC motor coupled to the shaft via rub belts. A radial load of 6000 lbs is applied onto the shaft and bearing by a spring mechanism. All bearings are force lubricated. Rexnord ZA-2115 double row bearings were installed on the shaft as shown in the Figure 2.1. PCB 353B33 High Sensitivity Quartz ICP accelerometers were installed on the bearing housing (two accelerometers for each bearing [x- and y-axes]). Sensor placement is also shown in Figure 1. All failures occurred after exceeding designed life time of the bearing which is more than 100 million revolutions.

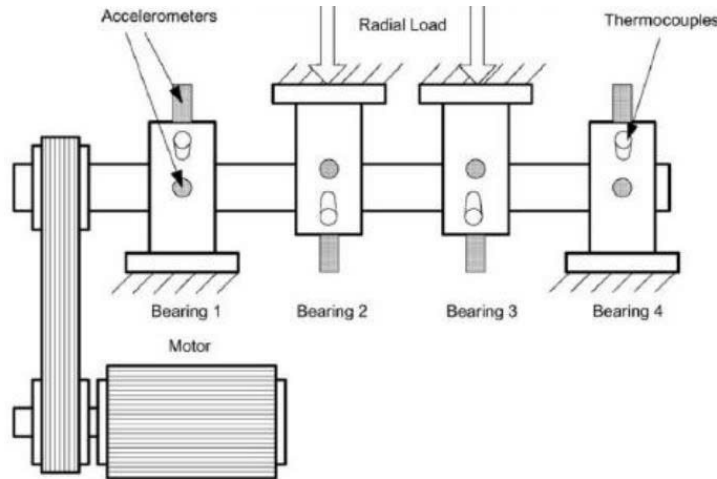


Figure 2.1: Dataset Experiment Setup

Each data set describes a test-to-failure experiment. Each data set consists of individual files that are 1-second vibration signal snapshots recorded at specific intervals. Each file consists of 20,480 points with the sampling rate set at 20 kHz. The file name indicates when the data was collected. Each record (row) in the data file is a data point. Data collection was facilitated by NI DAQ Card 6062E.

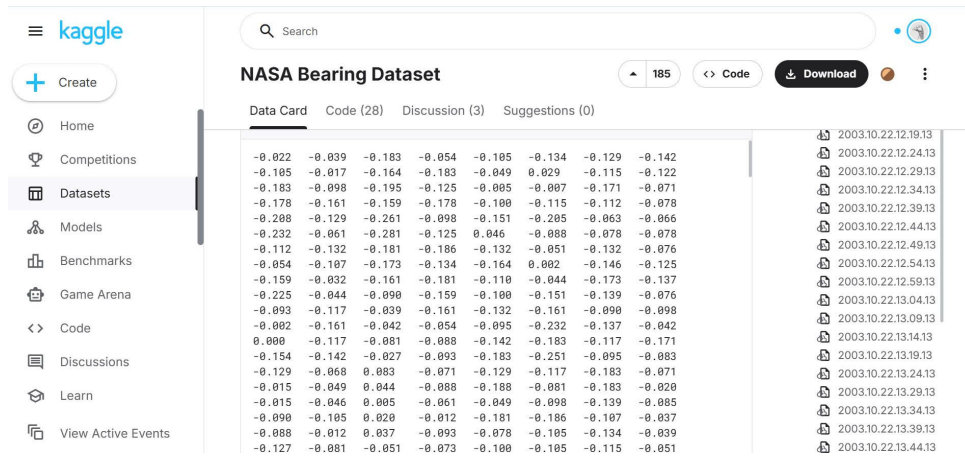


Figure 2.2: Dataset Overview

After considering the characteristic of Big Data, the reasons why we choose this dataset are:

- **Volume:** This dataset's size is about 6.53 GB. While 6.53 GB is small compared to industrial production scales, it is sufficient to demonstrate distributed processing. We believe that for this dataset size, it might not be processed effectively by traditional methods and so it needs a "big data" solution, which is the goal of the course.
- **Velocity:** This is the real challenge of this project. The dataset is the snapshot of the sensor in one second at rate 20 kHz, which mean there are 20,480 sample points captured in 1 seconds. Processing 20,480 points every second in real-time requires high-throughput stream processing (like Kafka/Spark) to prevent data loss.
- **Variety:** This dataset focuses on Volume/Velocity rather than Variety, which is a valid trade-off for this project. In this dataset, there are only raw accelerometer data of each bearing without any other type like image or sound. And the data source is just one source so we do not have the variety in the data. However in the future we may improve our system by adding sound of the bearing for better monitoring and predicting.

2.2 Feature Engineering

Secondly, we describe the process of transforming raw high-frequency sensor data into meaningful indicators of machine health. Our pipeline follows a two-stage approach: (1) Extraction of statistical time-domain features, and (2) Feature selection based on correlation analysis to reduce dimensionality and ensure computational efficiency for real-time processing.

2.2.1 Candidate Feature Extraction

The raw vibration signal consists of 20,480 data points per second. Feeding this high-dimensional raw data directly into a machine learning model is inefficient and noisy. Instead, we first extracted a comprehensive set of "Candidate Features" widely used in signal processing to capture different characteristics of the waveform:

- **Mean:** The average value of the signal amplitude. For vibration signals, this often centers around zero, but shifts can indicate DC offsets or sensor drift.

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

- **Root Mean Square (RMS):** Represents the continuous energy of the signal.

$$x_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

- **Standard Deviation (Std):** Measures the variability or "spread" of the signal from the mean. In zero-mean vibration data, this closely tracks the RMS and indicates overall vibration severity.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

- **Maximum Absolute Value (Max):** Identifies the single largest peak amplitude (shock) in the time window, often corresponding to the most severe impact event.

$$x_{\text{max}} = \max(|x_i|)$$

- **Peak-to-Peak (P2P):** Measures the total range of the vibration excursion from

the lowest trough to the highest peak.

$$x_{p2p} = \max(x) - \min(x)$$

- **Skewness:** Measures the asymmetry of the signal distribution. Healthy bearings typically have a symmetric distribution (skewness ≈ 0), while unilateral faults (like rubbing) can skew the distribution.

$$Sk = \frac{1}{N} \sum_{i=1}^N \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$$

- **Kurtosis:** Measures the "tailedness" or impulsiveness of the signal. A high kurtosis value indicates the presence of sharp, impulsive shocks (outliers), which is a key indicator of early-stage fatigue or cracking.

$$K = \frac{1}{N} \sum_{i=1}^N \left(\frac{x_i - \bar{x}}{\sigma} \right)^4$$

2.2.2 Correlation Analysis

After extracting, we validate the quality of our candidate features, we generated a Correlation Heatmap 2.3 across all 8 sensor channels.

From the heatmap, it points out some observation:

- **Redundancy Identification:** We observed a near-perfect positive correlation ($r > 0.95$, dark red blocks) between RMS and Standard Deviation. This is expected for zero-mean vibration signals. While mathematically redundant, we retained both to monitor both the energy (RMS) and the signal spread (Std Dev) for robustness against sensor drift.
- **Feature Independence:** All other features have low correlation with each other. This confirms that our features provides unique, non-redundant information about the bearings' health.

So after this step, our feature set contains mean, rms, std, max, p2p, kurtosis.

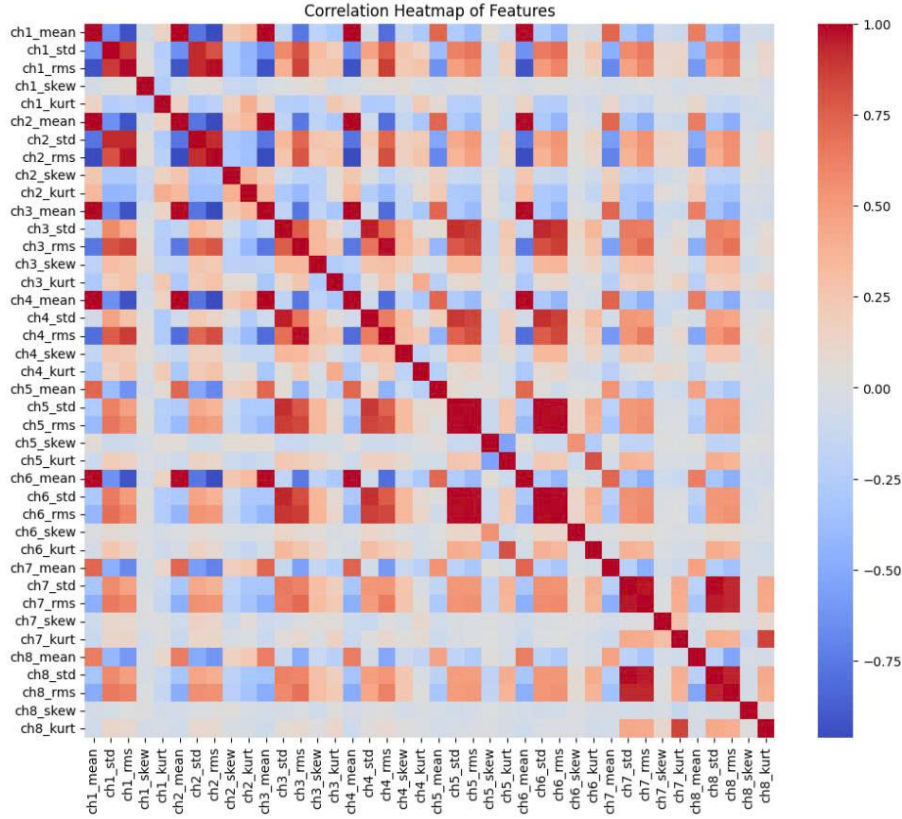


Figure 2.3: Dataset Overview

2.3 Predictive Model

Next, we discuss the model selection process, the theoretical reasons for using Gradient Boosting, and the training pipeline implemented to predict the Remaining Useful Life (RUL) of the bearings.

2.3.1 Candidate Models

To select the optimal approach for Remaining Useful Life (RUL) estimation, we evaluated some algorithms from classical statistical methods to modern deep learning architectures. Based on the characteristics of our dataset (time-series sensor data with limited failure instances), we narrowed our focus to three primary candidate models:

- **Linear Regression (Baseline Model):** We selected Linear Regression to serve as a performance baseline. It is the fundamental algorithm for regression tasks, modeling the relationship between sensor features and RUL as a straight line. While highly interpretable and computationally fast, it assumes a constant rate of degradation,

which may fail to capture the exponential "accelerated failure" phase often seen in bearing fatigue.

- **Long Short-Term Memory (LSTM):** LSTM is a specialized Recurrent Neural Network (RNN) designed specifically for sequential data.^[1] Unlike standard regression models, LSTMs possess internal "memory" cells that allow them to capture long-term temporal dependencies and historical trends in the vibration signal. However, LSTMs typically require massive datasets to generalize effectively and act as "black boxes," offering limited interpretability regarding the root cause of the prediction.
- **Gradient Boosting Regressor (GBR):** GBR is an ensemble machine learning technique that builds a prediction model by combining multiple weak learners (decision trees) in a sequential manner.^[2] It is particularly well-suited for this project because it handles non-linear relationships excellently and performs robustly on "small data" (datasets with few run-to-failure examples) where deep learning models might overfit. Additionally, GBR provides feature importance scores, enabling the diagnostic analysis required for our dashboard.

2.3.2 Model Selection

For this project, we selected the Gradient Boosting Regressor (GBR) as our primary predictive engine. We decided to choose this model for three key trade-offs:

- **Handling Non-Linear Degradation:** Bearing health typically follows a pattern where degradation is slow and linear for m , followed by an exponential acceleration in the final days. Linear regression fails to capture this non-linear behavior. GBR, as a non-linear ensemble method, naturally partitions the feature space to model both the steady state and the rapid failure state accurately.
- **Data Efficiency:** The NASA Bearing Dataset contains only 4 run-to-failure instances. Deep Learning models (like LSTMs) typically require thousands of examples to converge without overfitting. GBR is known to perform exceptionally well on smaller, tabular datasets ("Small Data" problems).
- **Interpretability (Diagnostic Capability):** Unlike Neural Networks, tree-based models provide Feature Importance scores. This allows our system to not just predict when a machine will fail, but also explain which signal (e.g., "Inner Race Kurtosis") is driving the risk. This supports the "Diagnostic" requirement of our dashboard.

2.3.3 Gradient Boosting Theory

Gradient Boosting is an ensemble machine learning technique that builds a strong predictive model by combining multiple "weak learners" (typically shallow decision trees) in a sequential manner.

Unlike Bagging algorithms (e.g., Random Forest) that build trees in parallel on different data subsets, Gradient Boosting builds trees iteratively. Each new tree is trained specifically to correct the errors made by the previous ensemble.

The goal is to construct a function $F(x)$ that minimizes a loss function $L(y, F(x))$, typically the Squared Error Loss $L = \frac{1}{2}(y - F(x))^2$.

Training Process:

1. **Initialization:** The model starts with a constant prediction $F_0(x)$, usually the mean of the target values.
2. **Iterative Improvement:** For each boosting stage $m = 1$ to M : The model calculates the pseudo-residuals (r_{im}), which represent the error of the current model. For Squared Error loss, this is simply the difference between the actual RUL and the predicted RUL:

$$r_{im} = \frac{1}{2}(y_i - F_{m-1}(x_i))^2$$

A new decision tree learner $h_m(x)$ is trained to predict these residuals (i.e., the tree learns the "shape" of the error). The model is updated by adding this new tree, scaled by a learning rate ν :

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

By iteratively fitting the residuals—effectively performing gradient descent in function space—the GBR model focuses its learning capacity on the "hard-to-predict" examples (the rapid failure stages), resulting in a highly accurate RUL estimation.

3 System architecture

In this section, we present the high-level architectural design of our predictive maintenance solution. The system is designed as a real-time data pipeline capable of ingesting high-frequency sensor streams, processing signal features on-the-fly, and delivering actionable insights to end-users with minimal latency.

3.1 Architecture Overview

To address the "Velocity" and "Volume" challenges of the NASA Bearing Dataset, we adopted a Stream Processing Architecture. This pipeline decouples data production from consumption, allowing our system to handle the 20 kHz sampling rate without bottlenecks.^[3]

The architecture consists of four distinct logical layers:

1. **Data Ingestion Layer:** Acts as a high-throughput buffer (Message Broker) to decouple data production from processing, ensuring zero data loss under load.
2. **Processing Layer:** A distributed compute engine that ingests raw streams, executes feature engineering logic in-memory, and performs model inference.
3. **Serving Layer:** A structured storage and visualization interface that surfaces insights to the maintenance dashboard.

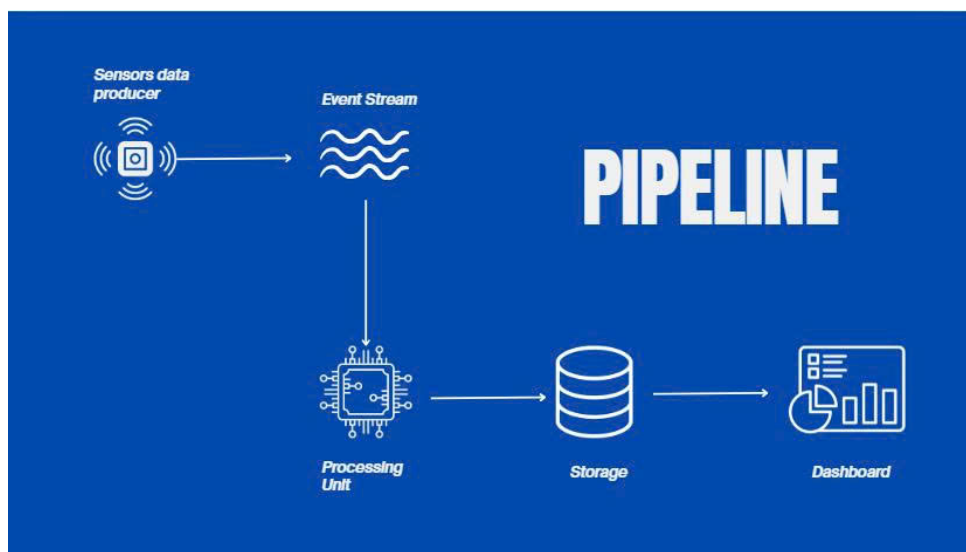


Figure 3.1: System Architecture

3.2 Detailed Module Design

3.2.1 Data Ingestion

Since the dataset is historical, we developed a simulation module that acts as a "Virtual Sensor." It reads the raw vibration archives (CSV) and replays them as a live stream.

To accelerate testing and demonstration, the system compresses the original timeline (10-minute intervals between files) into shorter windows (30 seconds), broadcasting a full snapshot of 20,480 sampling points per transmission. Data is serialized and pushed to a distributed message queue to decouple the simulation from the processing logic.

3.2.2 Processing Engine (The Core)

This layer acts as the "brain" of the architecture. It subscribes to the message queue and performs two critical transformation steps in memory:

Feature Engineering: The raw high-frequency signal (20,480 points) is aggregated into a low-dimensional feature vector (RMS, Kurtosis, etc.) using the statistical methods defined in Section 2.2.

Predictive Inference: The pre-trained Gradient Boosting Regressor (GBR) is loaded into the processing engine. As new feature vectors arrive, the model immediately calculates the Remaining Useful Life (RUL).

3.2.3 Storage Layer

Processed results—containing the timestamp, features, and predicted RUL—are persisted into a relational database. We utilized a structured SQL schema for the storage layer to ensure ACID compliance and to support complex analytical queries required by the reporting tools.

3.2.4 Visualization & Monitoring Layer

The final layer is the user interface for maintenance managers. The dashboard connects to the storage layer using a DirectvQuery mechanism. This ensures that the visualizations (gauges, health matrices) reflect the state of the machinery in near real-time, refreshing automatically as soon as new records are committed to the database.



4 Technology Stack

For each component, we evaluated several industry-standard options before selecting the tools that best fit the requirements of high-velocity sensor data processing and local reproducibility.

4.1 Data Ingestion

The data ingestion layer is responsible for streaming the NASA Bearing dataset as live sensor data into our system.

- **RabbitMQ/ActiveMQ:** While reliable for traditional messaging, these are not designed for the high-throughput requirements of continuous vibration data. They typically handle short-lived message delivery and lack the "true log" architecture needed for replaying historical data.
- **MQTT Brokers (e.g., Mosquitto):** Common in IoT environments due to their lightweight pub/sub nature, these brokers are not optimized for high-throughput analytics. Furthermore, they lack the tight integration with Spark Structured Streaming required for our processing layer.
- **Cloud Services:** These offer similar capabilities to our chosen stack but are cloud-vendor specific. They were deemed less suitable for this project, which prioritizes a local, Docker-based, and easily reproducible setup for academic evaluation.

Primary Technology: We selected **Apache Kafka** as our core ingestion engine because it is purpose-built for high-throughput streams. Its log-based architecture allows for easier data replay and retention compared to classic message queues. Crucially, Kafka offers native integration with Spark Structured Streaming, which enables a seamless transition from data ingestion to real-time processing.^[4]

4.2 Processing

The processing layer handles both the offline training of the predictive model and the online inference of real-time sensor streams.

- **Plain Python / Pandas:** Although excellent for exploratory data analysis, Pandas does not scale well to large datasets or continuous streams and lacks a built-in streaming engine.

- **Apache Hadoop MapReduce:** This legacy framework relies on disk-based processing, making it significantly slower than modern in-memory alternatives and more difficult to program for complex machine learning workflows.

Primary Technology: We utilized **Apache Spark** as a unified engine for both batch and streaming workloads. It provides the scale necessary to process 20,480 records per second and includes built-in connectors for Kafka (ingestion) and JDBC (storage). We leverage Spark to compute features such as RMS, Kurtosis, and Standard Deviation in real-time.^[5]

4.3 Storage

The storage layer acts as a serving layer and provides structured persistence for our predictions.

- **HDFS / MinIO:** These are suitable for raw file storage but are awkward for the ad-hoc SQL querying and joins required by Business Intelligence (BI) tools.
- **Cloud Warehouses:** These utilize pay-per-query models that can become expensive and introduce complexity into a local development environment.

Primary Technology: We chose **PostgreSQL** because it is an open-source, powerful relational database with strong support for analytical functions. It serves as our operational data store, providing a reliable SQL endpoint that integrates excellently with Power BI for visualization.^[6]

4.4 Dashboard

The final layer provides decision support through descriptive and predictive analytics.

- **Grafana:** While powerful for infrastructure monitoring, it is more DevOps-oriented and offers less flexibility for the complex data modeling and drag-and-drop analytics required for business stakeholders.
- **Tableau / Qlik:** These are top-tier BI tools but require expensive licenses and are not as tightly integrated into a Windows-based development environment as the Microsoft ecosystem.

Primary Technology: Power BI was selected for its familiar UI and powerful modeling capabilities. It allows us to visualize vibration history and Remaining Useful Life (RUL) predictions through interactive Fleet Health Matrices and risk logs.



4.5 Infrastructure and Deployment

To ensure the reproducibility and portability of our complex multi-service architecture (Kafka, Zookeeper, Spark, Postgres), we evaluated several deployment strategies.

- Bare Metal Installation: Installing each service directly on the host operating system (Windows/Linux) was rejected due to "Dependency Hell." Managing conflicting Java versions (e.g., Spark needs Java 8/11, while other tools might need newer versions) and environment variables across different developer machines is error-prone and time-consuming.
- Virtual Machines (VMs): While VMs provide strong isolation, they are resource-intensive. Running separate full operating systems for the Broker, Processor, and Database would introduce significant overhead, slowing down the development cycle on standard laptop hardware.
- Kubernetes (K8s): Although the industry standard for production orchestration, Kubernetes introduces excessive complexity for a local academic project. The overhead of managing a local cluster (Minikube/Kind) outweighs the benefits for a single-node demonstration.

Primary Technology: Docker & Docker Compose We selected Docker as our infrastructure backbone. It allows us to containerize each component (Kafka, Spark, Postgres) into lightweight, isolated environments that share the host kernel.^[7]

This allow us to reproduce the exact environment. We utilized `docker-compose.yml` to define the entire stack as code. This ensures that the system works identically on any machine with Docker installed, requiring only a single command (`docker-compose up`) to deploy the entire pipeline.

Also for networking, Docker Compose automatically creates a dedicated internal network, allowing our Spark container to resolve the Kafka broker and Postgres database by service name, simplifying configuration management.

5 Implementation

In this section, we outline the specific configuration and logic used to implement the architecture described in Section 3. The entire system was containerized using Docker to ensure reproducibility and consistency across development environments. In this section, we only demonstrate the core logic, not the full implementation. The full implementation can be accessed via Github in the Section 8 Video and Source Code

5.1 Data Ingestion

Infrastructure Configuration (Docker Compose): To deploy the streaming infrastructure, we utilized the `wurstmeister/kafka` image for the broker and `zookeeper` for cluster orchestration. The services were defined in `docker-compose.yaml` with explicit port mapping and volume mounting for persistence.

Note: For this academic implementation, we deployed the Kafka broker as a single-node cluster due to hardware resource constraints, though the configuration is fully compatible with multi-node scaling.

```
1 services:
2   zookeeper:
3     image: wurstmeister/zookeeper
4     ports:
5       - "2181:2181"
6
7   kafka:
8     image: wurstmeister/kafka
9     ports:
10      - "9092:9092"
11     environment:
12       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
13       KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
14       KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
15       KAFKA_CREATE_TOPICS: "data-sensors:1:1"
16       KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
17     volumes:
18       - /var/run/docker.sock:/var/run/docker.sock
19     depends_on:
20       - zookeeper
21
22   producer:
23     build: ./Producer
```



```
24 depends_on:
25     - kafka
26 volumes:
27     - ./NASA_Bearing_Data:/app/NASA_Bearing_Data
```

Listing 5.1: Docker Compose Configuration for Kafka

Producer Logic: The producer acts as a virtual sensor. It iterates through the historical dataset files, simulating a real-time feed. To mimic the physical behavior of the test rig, the producer reads a file (representing a 1-second snapshot) and streams the records to the `data-sensors` topic.

To simulate the 10-minute interval between sampling events in the original experiment, the producer introduces a latency period (`time.sleep()`) after efficiently flushing the buffer.

```
1 files = get_sorted_files(DATA_DIR)
2 print(f"Found {len(files)} files. Starting stream...")
3
4 for filename in files:
5     file_path = os.path.join(DATA_DIR, filename)
6
7     try:
8         # Load raw sensor data
9         df = pd.read_csv(file_path, sep='\t', header=None)
10        df.columns = ['b1a', 'b1b', 'b2a', 'b2b', 'b3a', 'b3b', 'b4a', 'b4b']
11    except Exception as e:
12        print(f"Skipping corrupt file {filename}: {e}")
13        continue
14
15    print(f"Streaming file: {filename} ({len(df)} rows)...")
16
17    # Stream individual sensor events
18    for index, row in df.iterrows():
19        message = {
20            "timestamp": timestamp_str,
21            "file": filename,
22            "b1": row['b1a'],
23            "b2": row['b2a'],
24            "b3": row['b3a'],
25            "b4": row['b4a']
26        }
27        producer.send(TOPIC_NAME, message)
28
```

```
29 # Flush buffer and simulate interval delay
30 producer.flush()
31 time.sleep(FILE_INTERVAL)
```

Listing 5.2: Python Producer Logic

5.2 Processing and Storage

Infrastructure Configuration: We use the image official Jupyter image and a script to submit the spark job.

```
1 # Use the official Jupyter image (Contains Python + Java + Spark)
2 FROM jupyter/pyspark-notebook:latest
3
4 WORKDIR /app
5
6 # Install dependencies from requirements.txt
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir --default-timeout=1000 -r requirements.
   txt
9
10 COPY preprocessor.py .
11 COPY start.sh .
12
13 # We use the start.sh file to submit the spark job.
14 CMD ["/start.sh"]
```

Listing 5.3: Python Producer Logic

5.2.1 Batch Processing (Training Model)

: This process happens before the whole application is deploy. It only used to train the predictive model. The process start from loading the entire dataset, train the model and then store it for the the streaming inference. Note that we derived the label RUL by subtracting the final timestamp of the crash bearing and the current timestamp.

```
1 def main():
2     set_feats = []
3     for i, path in enumerate(DATASET_PATHS, start=1):
4         print(f"[BEARING] Computing time features from {path}")
5         set_feats.append(time_features(path))
6
```

```
7  # Build supervised datasets \
8  sel_feats = ("max", "p2p", "rms", "std", "kurtosis")
9  supervised_dfs = [
10     build_rul_all_bearings(feats, selected_features=sel_feats) for
feats in set_feats
11 ]
12 # Train scikit-learn model
13 X = supervised_dfs[all_feature_cols].values
14 y = supervised_dfs["rul"].values
15
16 X_train, X_test, y_train, y_test = train_test_split(
17     X, y, test_size=0.2, random_state=42
18 )
19
20 model = GradientBoostingRegressor(
21     n_estimators=500,
22     learning_rate=0.05,
23     max_depth=3,
24     random_state=42,
25 )
26
27 print("[BEARING] Training GradientBoostingRegressor (scikit-learn)
...")
28 model.fit(X_train, y_train)
29
30 # Save model
31 MODELS_DIR.mkdir(parents=True, exist_ok=True)
32 model_path = MODELS_DIR / "bearing_rul_sklearn.pkl"
33 bundle = {
34     "model": model,
35     "feature_cols": all_feature_cols,
36 }
37 joblib.dump(bundle, model_path)
38 print(f"[BEARING] Saved scikit-learn model to {model_path}")
```

Listing 5.4: Training Model

5.2.2 Stream Processing And Storage

The core processing logic is implemented using Apache Spark Structured Streaming. The application connects to the Kafka broker, subscribing to the **data-sensors** topic.

We utilized a window-based aggregation strategy to transform the raw incoming rows

into the statistical feature vectors defined in Section 2. The data stream is grouped by file/timestamp, and calculate the RMS, Kurtosis, and Peak-to-Peak values in real-time.

First, we initialize the spark session. Then we define the structure schema of the data from event stream. After that we read the raw data from kafka topic, perform feature engineering on it and call `predict_bearing_batch()` function to predict the RUL and store it to Postgresql.

```
1 def main():
2     # 1. Initialize Spark Session
3     spark = SparkSession.builder \
4         .appName("Predictive Maintenance Consumer") \
5         .getOrCreate()
6
7     # 2. Define the Schema
8     schema = StructType([
9         StructField("timestamp", StringType(), True),
10        StructField("file", StringType(), True),
11        # Bearing data fields
12        StructField("b1", DoubleType(), True),
13        StructField("b2", DoubleType(), True),
14        StructField("b3", DoubleType(), True),
15        StructField("b4", DoubleType(), True),
16    ])
17    # 3. Read from kafka topic
18    raw_stream = spark.readStream \
19        .format("kafka") \
20        .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP_SERVERS) \
21        .option("subscribe", KAFKA_TOPIC) \
22        .option("startingOffsets", "earliest") \
23        .load()
24
25    # 4. Processing Data and Feature Engineering
26    feature_stream = raw_stream \
27        .withWatermark("event_time", "30 seconds") \
28        .groupBy("event_time") \
29        .agg(
30            first("timestamp").alias("timestamp"),
31
32            # b1
33            F.max(F.abs(col("b1"))).alias("b1_max"),
34            (F.max(F.abs(col("b1"))) + F.abs(F.min(col("b1")))).alias("b1_p2p"),
```



```
35     F.sqrt(F.avg(col("b1") * col("b1"))).alias("b1_rms"),
36     F.stddev(col("b1")).alias("b1_std"),
37     F.kurtosis(col("b1")).alias("b1_kurtosis"),
38
39     # b2
40     F.max(F.abs(col("b2"))).alias("b2_max"),
41     (F.max(F.abs(col("b2"))) + F.abs(F.min(col("b2")))).alias("
b2_p2p"),
42     F.sqrt(F.avg(col("b2") * col("b2"))).alias("b2_rms"),
43     F.stddev(col("b2")).alias("b2_std"),
44     F.kurtosis(col("b2")).alias("b2_kurtosis"),
45
46     # b3
47     F.max(F.abs(col("b3"))).alias("b3_max"),
48     (F.max(F.abs(col("b3"))) + F.abs(F.min(col("b3")))).alias("
b3_p2p"),
49     F.sqrt(F.avg(col("b3") * col("b3"))).alias("b3_rms"),
50     F.stddev(col("b3")).alias("b3_std"),
51     F.kurtosis(col("b3")).alias("b3_kurtosis"),
52
53     # b4
54     F.max(F.abs(col("b4"))).alias("b4_max"),
55     (F.max(F.abs(col("b4"))) + F.abs(F.min(col("b4")))).alias("
b4_p2p"),
56     F.sqrt(F.avg(col("b4") * col("b4"))).alias("b4_rms"),
57     F.stddev(col("b4")).alias("b4_std"),
58     F.kurtosis(col("b4")).alias("b4_kurtosis"),
59 )
60 # 5. Call predict_bearing_batch for prediction and storage
61 query = feature_stream.writeStream \
62     .outputMode("append") \
63     .foreachBatch(predict_bearing_batch) \
64     .start()
```

Listing 5.5: Spark Streaming Logic

This is our a simplified version predicting_bearing_batch() function, we first load the model, perform inference and finally write the result to the database through JDBC.

```
1 bearing_model_path = "/app/models/bearing_rul_sklearn.pkl"
2
3 # Load the model
4 print(f"[STREAM] Loading bearing model from {bearing_model_path}")
5 bearing_bundle = joblib.load(bearing_model_path)
```

```
6 bearing_model = bearing_bundle["model"]
7 bearing_feature_cols = bearing_bundle["feature_cols"]
8 print(f"[STREAM] Bearing model loaded. Features: {bearing_feature_cols}"
9     )
10
11 def predict_bearing_batch(df, batch_id: int):
12     for bearing_idx in range(1, 5):
13         pdf[f"b{bearing_idx}_rul"] = bearing_model.predict(X_bearing)
14
15     final_df = spark.createDataFrame(pdf)
16
17     try:
18         final_df.select(
19             "timestamp",
20
21             # b1
22             "b1_max", "b1_p2p", "b1_rms", "b1_std", "b1_kurtosis", "
23             b1_rul",
24
25             # b2
26             "b2_max", "b2_p2p", "b2_rms", "b2_std", "b2_kurtosis", "
27             b2_rul",
28
29             # b3
30             "b3_max", "b3_p2p", "b3_rms", "b3_std", "b3_kurtosis", "
31             b3_rul",
32
33             # b4
34             "b4_max", "b4_p2p", "b4_rms", "b4_std", "b4_kurtosis", "
35             b4_rul",
36         ) \
37         .write \
38         .mode("append") \
39         .jdbc(url=JDBC_URL, table="bearing_predictions", properties=
40             JDBC_PROPS)
41
42     print("Successfully wrote to Database!")
43 except Exception as e:
44     print(f"Database Error: {e}")
```

Listing 5.6: Model Inference and Storage



5.3 Dashboard Integration

The visualization layer is implemented in Power BI, connecting to the PostgreSQL database via the standard JDBC connector. We utilized the DirectQuery mode rather than Import mode. This configuration ensures that the dashboard does not cache data; instead, it queries the database every time a visual is refreshed or interacted with, providing a near real-time view of the fleet's health as new predictions are written by the Spark engine. This implementation will be discussed further in the System Demonstration Section.

6 System Demonstration & Results

Finally we move to the system demonstration and result. In this part, we will illustrate our dashboard, the final product that our customer use. We will discuss about the dashboard and features.

6.1 Overview Page

The Overview Page is designed as the primary entry point for High-Level Operations Managers. The design philosophy for this view is "Management by Exception"—prioritizing immediate visibility of critical risks over raw technical data. The goal is to answer the fundamental operational question in under 5 seconds: *"Is the fleet healthy, or is immediate intervention required?"*



Figure 6.1: Executive summary page

6.1.1 Feature 2: Executive Summary

- **Overall Health Gauge:** This visual aggregates the condition of the entire fleet into a single metric. It is calculated as the weighted average of the Remaining Useful Life (RUL) across all active sensors. A score below 30% (indicated in red) triggers a visual alarm, signaling systemic degradation.
- **Normalized Health Scores (0-100%):** Instead of displaying the raw "Predicted Hours" (which can be ambiguous depending on the bearing type), we implemented

a **Normalized Health Score**.

$$\text{Health Score} = \left(\frac{\text{Predicted RUL}}{\text{Max Design Life}} \right) \times 100$$

This design choice reduces cognitive load for the manager. A "10%" score universally indicates a critical state, regardless of whether the bearing's max life is 1,000 hours or 10,000 hours, allowing for standardized risk assessment.

- **Operational Counters:** The bottom panel provides an instant inventory snapshot, displaying the **Total Sensors** monitored (4) and the count of **Active Alerts** (3). This explicitly quantifies the maintenance backlog, allowing managers to allocate resources effectively.

6.1.2 Feature 2: Fleet Health Matrix (Diagnostic Scatter Plot)

While the Executive Summary provides a high-level status, the **Fleet Health Matrix** (Figure 6.1, Right Panel) is designed for **Reliability Engineers** who require granular diagnostic context.

This visualization moves beyond simple single-variable analysis by correlating two critical dimensions of machine health:

- **Y-Axis (Severity):** The current Root Mean Square (RMS) amplitude, representing the *present* physical vibration energy.
- **X-Axis (Urgency):** The predicted Remaining Useful Life (RUL), representing the *future* time-to-failure.

Strategic Value: Multi-Dimensional Triage By plotting these two variables against each other, the matrix segments the fleet into distinct **Operational Quadrants**, enabling more sophisticated decision-making than simple threshold alerts:

- **High RMS / Low RUL (Top-Left):** This quadrant represents **Immediate Criticality**. Bearings in this zone (e.g., the Orange point in the figure) exhibit both high physical vibration and a model prediction of imminent failure. These assets require emergency shutdown and replacement.
- **Low RMS / Low RUL (Bottom-Left):** This represents **Silent Failure Risks**. A bearing might not yet be vibrating violently (Low RMS) but is showing statistical

signs of fatigue (Kurtosis/Skewness) that the AI model has detected as nearing end-of-life. This insight prevents "surprise" failures that traditional RMS monitoring would miss.

6.2 Vibration History and RUL Prediction

While the Overview page identifies serve High Level Manager, this Vibration History and RUL Prediction (Figure 6.2) serves the engineering workbench for determining *why* and *how* the failure is progressing. It also help them schedule the next maintenance effectively based on the predicted RUL.

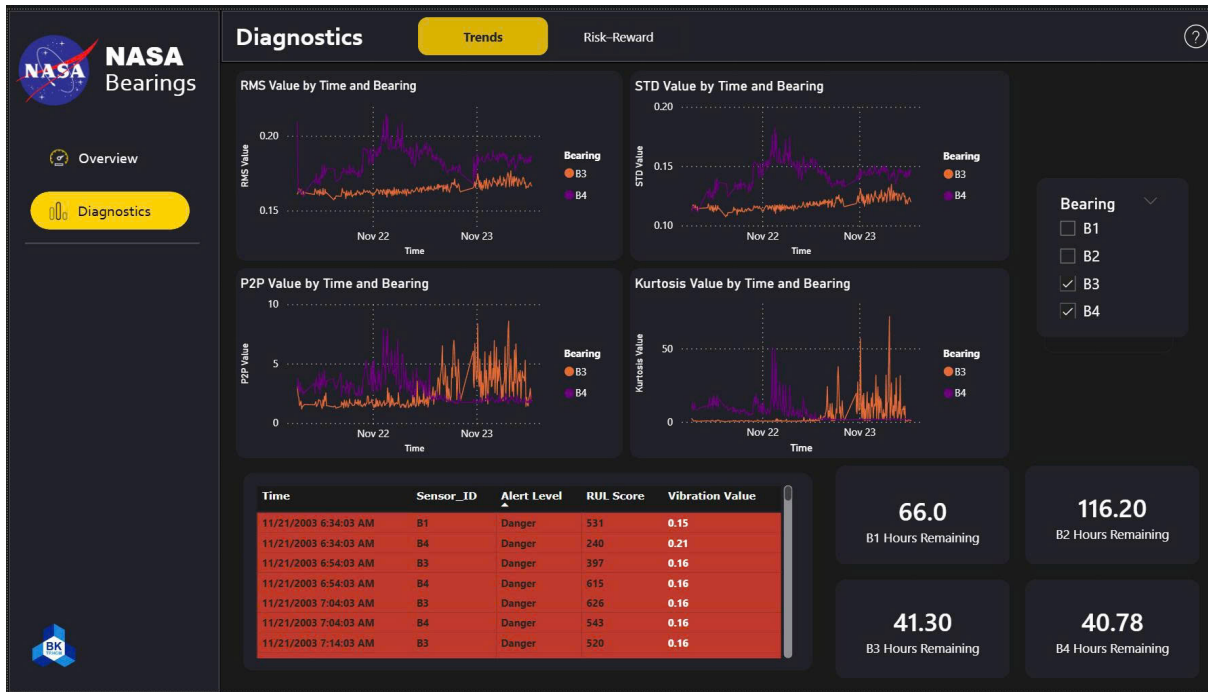


Figure 6.2: Vibration History and RUL Page

6.2.1 Feature 3: Vibration History and Trend

The core value of this page is a time-series view of the sensor data. It provides the "proof" behind the alert, allowing technicians to validate why the AI triggered an alarm before they stop the machine.

In this page, the top left section shows the history value of four features: RMS, STD, P2P and Kurtosis. As seen in the figure, Bearings 3 and 4 show a gradual energy increase, indicating loosening or late-stage wear. The "Bearing" slicer (Right Panel) allows users to

overlay multiple signals (e.g., B3 vs. B4) to distinguish between a systemic issue (affecting all bearings) and an isolated component failure. And we can also compare two bearings and identify if some bearings tend to degrade faster and take action.

6.2.2 Alert History Log

The Alert Log Table (Bottom Left) functions as a searchable, historical audit trail of the system's detected anomalies. It displays the last N (e.g., 20) critical events with precise columns: *Timestamp*, *Sensor_ID*, *Alert_Level*, *Vibration_Value*, and *RUL_Estimate*.

- **Pattern Recognition:** This tabular allow users to do some diagnostic on the system. For example, if Bearing 3 triggers a "Danger" alert consistently every Tuesday at 9:00 AM, a Reliability Engineer can correlate this pattern with external factors (e.g., a specific weekly high-load process) rather than assuming random component failure.
- **Post-Mortem Analysis:** It serves as a "Black Box" recorder, allowing teams to reconstruct the timeline of degradation after a failure occurs to improve future predictive models.

6.2.3 Feature 5: Remaining Useful Life (RUL) Gauge

Target User: Production Planners / Schedulers The RUL Gauge (Bottom Right) acts as a simplified "Countdown Clock" for operational decision-making. Unlike the raw trend lines used by engineers, this visualization answers the critical business question: "*When should we do the maintenance?*"

6.3 Prescriptive Analytics: Risk-Reward Optimization

While the Diagnostics page provides the technical "When," the Risk-Reward Page (Figure 6.3) answers the financial "What should we do?" This module moves beyond pure prediction to Prescriptive Analytics, helping managers balance the economic pressure of production against the financial risk of catastrophic failure.

6.3.1 Feature 6: Effective Plan

Target User: Plan Managers / Financial Planners

In real-world manufacturing, maintenance decisions are rarely binary. A machine with only 48 hours of RUL might be kept running if it is producing high-value batch orders,

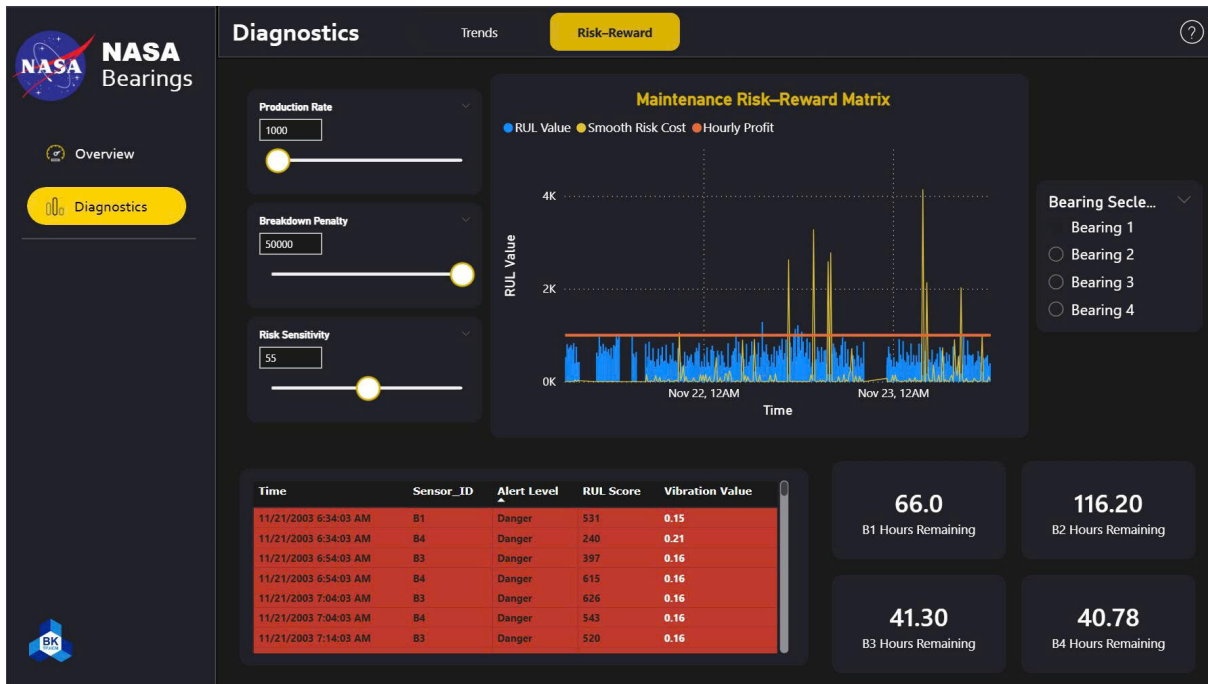


Figure 6.3: Maintenance Risk-Reward Matrix for Economic Optimization

provided the risk is calculated. This module allows managers to input dynamic business variables to simulate different scenarios:

- Dynamic Business Inputs (Sliders):** The dashboard allows the user to define the economic context of the machine:
 - **Production Rate:** Represents the hourly revenue generated by the machine (Value of keeping it running).
 - **Breakdown Penalty:** Represents the total cost of an unplanned crash (Replacement parts + Labor + Safety fines).
 - **Risk Sensitivity:** A weighting factor that adjusts how "conservative" the system should be. A higher sensitivity shifts the decision boundary earlier, prioritizing asset safety over profit.
- The Risk-Cost Function:** The central visualization (Figure 6.3, Center Panel) plots the **Smooth Risk Cost** (Yellow Line) against the **RUL** (Blue Bars). The system calculates a "Risk Cost" using the formula:

$$\text{Risk Cost} = P(\text{Failure}) \times \text{Breakdown Penalty}$$

As the RUL decreases (Blue bars drop), the Probability of Failure $P(\text{Failure})$ increases exponentially.

. This visualization empowers managers to make data-driven financial decisions rather than guessing. Therefore, whenever the risk cost exceed the profit, which mean the yellow line is higher than the orange line, we should stop the machine immediately and do maintenance.

Scenario Analysis - High Value vs. High Risk:

- **Scenario A (High Profit):** If the "Production Rate" is set high (e.g., a rush order), the system might recommend delaying maintenance because the immediate profit outweighs the statistical risk of a crash.
- **Scenario B (High Penalty):** If the "Breakdown Penalty" is high (e.g., the machine is critical and has no spare parts), the system will recommend an early stop even if the RUL is still moderate, prioritizing asset protection.

Business Value: This module solves the "Run-to-Failure" dilemma. It allows the company to squeeze the maximum value out of an asset (Run it as long as profitable) but stops exactly before the financial risk becomes unacceptable, maximizing the **Return on Assets (ROA)**.

7 Conclusion & Future Work

7.1 Conclusion

In this project, we successfully designed and implemented an end-to-end Predictive Maintenance System capable of processing high-velocity vibration data to forecast the Remaining Useful Life (RUL) of rotating machinery. By leveraging a modern Big Data technology stack—comprising **Apache Kafka** for ingestion, **Apache Spark** for distributed stream processing, **PostgreSQL** for storage and **Gradient Boosting** for inference, we addressed the three core challenges of the NASA Bearing Dataset such as Velocity, Volume and Value. We also provide the users with the dashboard with several features from descriptive, diagnostic, predictive and prescriptive analytics.

7.2 Limitations

While the system meets the functional requirements of the capstone project, we acknowledge specific limitations inherent to the academic scope:

- **Infrastructure Constraints:** The current deployment operates on a single-node Docker environment. While the logic is distributed (Spark/Kafka), the physical throughput is limited by the host machine's resources, preventing true horizontal scaling testing.
- **Model Generalization:** The predictive model was trained on a specific set of run-to-failure experiments. In a real-world scenario, the model would need to be re-trained or fine-tuned to handle different bearing types, load conditions, and variable rotational speeds.

7.3 Future Work

To evolve this prototype into a production-grade industrial solution, we propose the following enhancements:

1. **Cluster Deployment:** As we already deploy everything on Docker, it would be easier to employ cluster managing system such as Kubernetes to achieve the production scale performance.
2. **Deep Learning Integration:** We intend to explore Long Short-Term Memory (LSTM) or Transformer networks. These architectures can learn temporal depen-



dencies directly from raw waveforms, potentially eliminating the need for manual feature engineering and improving accuracy on larger datasets.

3. **Closed-Loop Automation:** Future iterations could integrate directly with the factory's PLC (Programmable Logic Controller) systems to automatically trigger a "Soft Stop" or reduce motor speed when the "Risk-Reward" metric exceeds a critical safety threshold, creating a fully autonomous self-protecting machine.

We gained invaluable practical experience through this project. Thank you for your advice and support, and we wish you all the best.



8 Video and Source Code

The source code is accessible via Github: https://github.com/chaunguye/predictive_maintenance

The video demo is uploaded on Youtube: <https://youtu.be/HJ4Glhi1eKw>



References

- [1] GeeksforGeeks, “What is LSTM - Long Short Term Memory?.” <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>, December 2025. Updated: 23 Dec 2025.
- [2] B. T. Tung, “Gradient boosting - Tất tần tât về thuật toán mạnh mẽ nhất trong Machine Learning.” <https://viblo.asia/p/gradient-boosting-tat-tan-tat-v-e-thuat-toan-manh-me-nhat-trong-machine-learning-YWOZrN7vZQ0>, May 2021.
- [3] W. Hammoudi, “Building a scalable data pipeline: A step-by-step guide with Kafka, Spark, PostgreSQL, and Dash.” <https://medium.com/towardsdev/building-a-scalable-data-pipeline-a-step-by-step-guide-with-kafka-spark-postgresql-and-dash-e239c1ab9e36>, November 2024.
- [4] The Apache Software Foundation, “Apache kafka 4.1 documentation - introduction.” <https://kafka.apache.org/41/getting-started/introduction/>, 2025. Accessed: 2026-01-01.
- [5] The Apache Software Foundation, “Apache spark documentation.” <https://spark.apache.org/docs/latest/>, 2025. Accessed: 2026-01-01.
- [6] PostgreSQL Global Development Group, “Postgresql documentation.” <https://www.postgresql.org/docs/>, 2025. Accessed: 2026-01-01.
- [7] Docker Inc., “Docker compose overview.” <https://docs.docker.com/compose/gettingstarted/>, 2025. Accessed: 2026-01-01.